



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Laboratory Record

MAS5001 Artificial Intelligence

Slot: C14 + E11 + E12

Class Number: BL2026270100787

Fall Semester: 2026-2027

Venue: AB02-201

Name of the student: Shiva Kumar

Register Number: 26MAS10029

Index

Exp. No.	Name of the Experiment	Page Number
1.	Depth-First Search and Breadth-First Search Algorithm	3
2.	Building an 8-puzzle solver using A-star search algorithm	6
3.	Solving Travelling Salesman problem using evolutionary search algorithm	9
4.	Implementation of First Order Inductive Learner (FOIL) for Parsing a family tree	
5.	Build the Tower of Hanoi puzzle using AI	
6.	Develop a game where play strategies are based on minmax search procedure.	
7.	Building Simple expert system	
8.	Perform an AI system test by developing an application.	
9.	References	

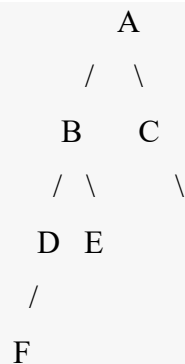
Exp No: 1	Depth-First Search and Breadth-First Search Algorithm
23-07-2026	

Aim:

To design and implement Depth-First Search and Breadth-First Search algorithm for traversing a graph having 12 nodes.

Objective:

1. To write a program to implement Depth-First Search for traversing a graph having 12 nodes.
2. To write a program to implement Breadth-First-Search for traversing a graph having 12 nodes.



Program Code:

DFS ALGORITHM

```

from collections import defaultdict

# Graph class
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    # Add edge to the graph (undirected by default)
    def add_edge(self, u, v):
        self.graph[u].append(v)

```

```

        self.graph[v].append(u) # Remove this line for directed graph

# Depth-First Search (DFS)
def dfs(self, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    print(start, end=' ')

    for neighbor in self.graph[start]:
        if neighbor not in visited:
            self.dfs(neighbor, visited)

# Example usage
g = Graph()

g.add_edge('A', 'B')
g.add_edge('A', 'C')
g.add_edge('B', 'D')
g.add_edge('B', 'E')
g.add_edge('C', 'F')
g.add_edge('E', 'F')

print("DFS Traversal:")
g.dfs('A')

```

Output:

```

DFS Traversal:
A B D E F C

```

BFS ALGORITHM

```
from collections import deque, defaultdict
# Graph class
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)
    # Add edge to the graph (undirected by default)
    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.graph[v].append(u) # Remove this line for directed graph

    # Breadth-First Search
    def bfs(self, start):
        visited = set()
        queue = deque([start])
        visited.add(start)
        while queue:
            vertex = queue.popleft()
            print(vertex, end=' ')
            for neighbor in self.graph[vertex]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('B', 'E')
    g.add_edge('C', 'F')
    g.add_edge('E', 'F')
    print("\nBFS traversal starting from A:")
    g.bfs('A')
```

Output:

```
BFS traversal starting from A:
A B C D E F
```

COLAB LINK: https://colab.research.google.com/drive/1Y6-PBkw2eMVIkjN_hiDv-atWYt6vrfRi#scrollTo=p8Nwoy7dYDvY

Experiment No. 2:	<i>Build an 8-Puzzle Solver Using a Search Algorithm</i>
24-07-2026	

Aim:

To design and implement an 8-Puzzle Solver using the A* (A-Star) Search Algorithm for finding the shortest path from the initial state to the goal state.

Objectives:

1. To understand the working of the A* Search Algorithm.
2. To implement the 8-Puzzle Solver using Python.
3. To find the optimal solution from the initial state to the goal state.
4. To evaluate the efficiency of heuristic search.

Program Code:

```
import heapq
# Manhattan distance heuristic
def manhattan_distance(state, goal):
    distance = 0
    for i in range(1, 9): # tiles 1-8
        xi, yi = divmod(state.index(i), 3)
        xg, yg = divmod(goal.index(i), 3)
        distance += abs(xi - xg) + abs(yi - yg)
    return distance
# Get possible moves
def get_neighbors(state):
    neighbors = []
    idx = state.index(0) # position of blank
    x, y = divmod(idx, 3)
    moves = [(-1,0), (1,0), (0,-1), (0,1)] # up, down, left, right
    for dx, dy in moves:
```

```

        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            n_idx = nx * 3 + ny
            new_state = list(state)
            new_state[idx], new_state[n_idx] = new_state[n_idx],
new_state[idx]
            neighbors.append(tuple(new_state))
        return neighbors
# A* search for 8-puzzle
def a_star(start, goal):
    pq = []
    heapq.heappush(pq, (manhattan_distance(start, goal), 0, start,
None))
    visited = {}
    while pq:
        f, g, state, parent = heapq.heappop(pq)
        if state in visited:
            continue
        visited[state] = parent
        if state == goal:
            path = []
            while state:
                path.append(state)
                state = visited[state]
            return path[::-1]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                heapq.heappush(pq, (g+1 + manhattan_distance(neighbor,
goal), g+1, neighbor, state))

    return None
# Example usage
start_state = (1, 2, 3, 4, 0, 6, 7, 5, 8)
go_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)
solution = a_star(start_state, go_state)
# Print solution path
if solution:
    for step in solution:
        print(f"{step[0:3]}\n{step[3:6]}\n{step[6:9]}\n")
else:
    print("No solution found")

```

OUTPUT:

```
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

COLAB LINK:

<https://colab.research.google.com/drive/13YFynVzPEaqqJ-3xRr1doUUvYihzIOd#scrollTo=R-CTRbTcP4Z>

Experiment No. 3	Solving Travelling Salesman Problem Using Evolutionary Search Algorithm
24-07-2026	

Aim :

To design and implement a solution for the Travelling Salesman Problem (TSP) using an Evolutionary Search Algorithm (Genetic Algorithm) to find the shortest possible route that visits each city exactly once and returns to the starting city.

Objectives :

1. To understand the Travelling Salesman Problem (TSP).
2. To implement an Evolutionary Search Algorithm (Genetic Algorithm) in Python.
3. To find an optimal or near-optimal route for visiting all cities.
4. To analyze the performance of the Genetic Algorithm in solving optimization problems.

Solving Travelling Salesman problem using evolutionary search algorithm.

Program Code:

```
import random
import math

# -----
# Helper functions
# -----

# Distance between two points
def distance(city1, city2):
    return math.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)

# Total distance of a tour
def total_distance(tour, cities):
```

```

dist = 0
for i in range(len(tour)):
    city_a = cities[tour[i]]
    city_b = cities[tour[(i + 1) % len(tour)]]
    dist += distance(city_a, city_b)
return dist

# Fitness = inverse of distance (shorter tour = higher fitness)
def fitness(tour, cities):
    return 1 / total_distance(tour, cities)

# -----
# GA operators
# -----

# Tournament selection
def selection(population, cities, k=3):
    selected = random.sample(population, k)
    selected.sort(key=lambda t: total_distance(t, cities))
    return selected[0]

# Ordered Crossover (OX)
def crossover(parent1, parent2):
    size = len(parent1)
    start, end = sorted(random.sample(range(size), 2))
    child = [None] * size

    # Copy slice from parent1
    child[start:end+1] = parent1[start:end+1]

    # Fill remaining from parent2
    p2_index = 0
    for i in range(size):
        if child[i] is None:
            while parent2[p2_index] in child:
                p2_index += 1
            child[i] = parent2[p2_index]
    return child

# Swap mutation
def mutate(tour, mutation_rate=0.01):

    for i in range(len(tour)):

```

```

        if random.random() < mutation_rate:
            j = random.randint(0, len(tour)-1)
            tour[i], tour[j] = tour[j], tour[i]
    return tour

# -----
# Main GA function
# -----

def genetic_algorithm_tsp(cities, population_size=100, generations=500,
mutation_rate=0.02):
    # Initial random population
    population = [random.sample(range(len(cities)), len(cities)) for _ in
range(population_size)]

    best_tour = min(population, key=lambda t: total_distance(t, cities))
    best_distance = total_distance(best_tour, cities)

    for gen in range(generations):
        new_population = []
        for _ in range(population_size):
            parent1 = selection(population, cities)
            parent2 = selection(population, cities)
            child = crossover(parent1, parent2)
            child = mutate(child, mutation_rate)
            new_population.append(child)

        population = new_population

        # Track best tour so far
        current_best = min(population, key=lambda t: total_distance(t,
cities))
        current_best_dist = total_distance(current_best, cities)
        if current_best_dist < best_distance:
            best_tour, best_distance = current_best, current_best_dist

        if gen % 50 == 0:
            print(f"Generation {gen} -> Best Distance:
{best_distance:.2f}")

    return best_tour, best_distance

# -----

```

```

# Example run
# -----

if __name__ == "__main__":
    # Example: 10 cities with (x, y) coordinates
    cities = [(random.randint(0, 100), random.randint(0, 100)) for _ in
range(10)]

    print("Cities:", cities)
    best_tour, best_distance = genetic_algorithm_tsp(cities,
population_size=200, generations=500,
                                                    mutation_rate=0.02)

    print("\nBest Tour:", best_tour)
    print("Best Distance:", best_distance)

```

OUTPUT:

```

Cities: [(60, 97), (63, 93), (57, 0), (36, 80), (86, 39), (22, 98), (33,
47), (78, 39), (54, 74), (46, 49)]
Generation 0 -> Best Distance: 323.08
Generation 50 -> Best Distance: 282.39
Generation 100 -> Best Distance: 282.39
Generation 150 -> Best Distance: 282.39
Generation 200 -> Best Distance: 282.39
Generation 250 -> Best Distance: 282.39
Generation 300 -> Best Distance: 282.39
Generation 350 -> Best Distance: 282.39
Generation 400 -> Best Distance: 282.39
Generation 450 -> Best Distance: 282.39

Best Tour: [9, 2, 4, 7, 8, 1, 0, 5, 3, 6]
Best Distance: 282.3876096373317

```

COLAB LINK:

<https://colab.research.google.com/drive/1oLpO9HYHOMKWWRMufb9jblC4x0q2uFpj?usp=sharing#scrollTo=ExoVH-WKb44A>

